

α Sentiment Analysis

Testing Whether Corporate Disclosure Language Predicts Future Stock Returns

1 Research Question

Most stock-market prediction projects begin with prices, technical indicators, or accounting ratios. This project starts from a different idea: public companies are required to explain their own businesses in detail every year. A Form 10-K report is not marketing copy. It is a regulated document in which management describes business risks, operating results, legal exposure, competition, liquidity, market conditions, and future uncertainty. If a company's outlook is quietly deteriorating, some trace of that deterioration may appear in the way the company writes about itself before it becomes obvious in simple financial ratios.

The narrative of this project is therefore simple and testable. First, collect annual 10-K reports from SEC EDGAR. Second, turn the language in those reports into measurable signals: negativity, uncertainty, litigation language, readability, document length, and year-over-year textual change. Third, link each filing to future stock returns after the actual filing date. Finally, test whether these textual signals help predict which stocks later outperform or underperform the market.

The project is intentionally designed around a scientific research question: can textual features extracted from annual 10-K reports predict future stock returns after controlling for simple market and firm-level baselines? The emphasis is on testing this hypothesis carefully rather than assuming that textual signals must be useful. In addition to future returns, the project also examines related financial outcomes such as future volatility, market-adjusted performance, and the difference between short-term and long-term return horizons.

This research question builds on a literature showing that corporate filings can contain measurable information beyond the numerical statements alone. Loughran and McDonald [1] show why finance-specific dictionaries are necessary for 10-K analysis, because many words that appear negative in general-purpose dictionaries have neutral meanings in accounting and finance. Jegadeesh and Wu [2] further show that the tone of 10-K filings can be connected to market reactions when words are weighted by their empirical relation to returns. Cohen, Malloy, and Nguyen [3] provide a closely related rationale for examining year-over-year textual change: if firms alter the language of their disclosures, those changes may reveal information about future operations that investors do not fully process immediately. Following this line of work, the project examines not only future returns, but also volatility, market-adjusted performance, and differences across return horizons, because disclosure language may indicate both expected performance and uncertainty around that performance.

2 Data Sources

The project uses two main data sources. The 10-K filings and filing metadata are downloaded from SEC EDGAR, which provides the annual reports, filing dates, CIK identifiers, accession

numbers, and document URLs. The financial market data are downloaded with `yfinance`, which provides adjusted stock prices and the SPY benchmark prices used to calculate forward returns, excess returns, and realized volatility.

3 EDGAR Download Procedure

The first operational step of the project is to build a reproducible pipeline for downloading annual 10-K filings from SEC EDGAR. EDGAR does not require an account or an API key, but automated scripts must identify themselves with a user-agent string. In practice, this means setting an environment variable in the terminal before running the downloader:

```
export SEC_USER_AGENT="your name your.email@example.com"
```

The command `export` creates a temporary environment variable for the current terminal session. The variable name `SEC_USER_AGENT` is read by the Python script and sent with each request to the SEC. The value should contain a name and email address so that the script follows SEC access guidelines. The command only affects the active terminal window, if the terminal is closed, the variable must be set again.

The downloader is implemented in `Code/edgar_10k_download.py`. Its purpose is to transform a list of stock tickers into locally stored 10-K filings and a metadata table. The script is divided into small functions because the EDGAR pipeline has several distinct tasks: creating the SEC connection, requesting data, mapping tickers to company identifiers, finding annual filings, downloading documents, and saving the result in a form that later analysis code can use.

The function `sec_session()` starts the process. It reads `SEC_USER_AGENT` from the terminal environment and creates a `requests.Session`. A session is useful because the same HTTP settings can be reused for every request. In this project, that means each SEC request is sent with the same user-agent string, so the script behaves like one identifiable research client.

The helper functions `sec_get()`, `sec_get_json()`, and `sec_get_bytes()` handle communication with SEC servers. `sec_get()` is the central request function. It waits between requests, sends the URL, checks the server response, and retries temporary errors such as 429, 500, 502, 503, and 504. This retry logic matters because large EDGAR downloads can be interrupted by temporary server load or rate limits. `sec_get_json()` is used when the SEC response should become structured Python data, while `sec_get_bytes()` is used when the script downloads the raw filing document itself.

The first SEC data source used by the script is the company ticker mapping:

```
https://www.sec.gov/files/company_tickers.json
```

The function `load_ticker_map()` downloads this file and converts it into a `pandas` table. This table connects stock tickers to CIK numbers. A CIK is the permanent company identifier used by the SEC. Because EDGAR's filing-history endpoint is organized by CIK rather than by ticker, this mapping step is necessary before any 10-K documents can be found. The function `cik_for_ticker()` then looks up one ticker in this table and returns the corresponding CIK. After the ticker has been mapped to a CIK, the script requests the company's submission history from:

```
https://data.sec.gov/submissions/CIK{cik}.json
```

This step is handled by `recent_10k_filings()`. The returned JSON file contains recent filings, including the form type, filing date, accession number, report date, and primary document name. The function converts this JSON structure into a `pandas` table and filters it to keep only annual report forms, specifically 10-K and 10-K/A. The command-line argument `--limit`

controls how many of these annual filings are kept for each company.

For each selected filing, `recent_10k_filings()` also constructs the direct SEC archive URL from three pieces: the CIK, the accession number, and the primary document name. The accession number identifies the filing submission, while the primary document name identifies the actual HTML document inside that submission. This URL is passed to `download_filing()`, which writes the filing document to `data/raw/filings/`. If the file already exists locally, `download_filing()` keeps the existing copy instead of downloading it again. This behavior is important for long runs because the script can be restarted without re-downloading filings that are already saved.

For the full research sample, typing every ticker into the terminal is not practical. Instead, the project uses a ticker-universe file. This is a plain text file with one ticker per line. A small example file is stored at:

```
config/tickers_sample.txt
```

The downloader can read this file directly:

```
python Code/edgar_10k_download.py \  
--tickers-file config/tickers_sp500.txt \  
--limit 10 \  
--metadata-output data/interim/filings_metadata_sp500.csv \  
--sleep-seconds 0.75 \  
--max-retries 8
```

The backslash at the end of the first line is a shell line-continuation character. It allows one long command to be written across several lines. The argument `--tickers-file` tells the script to read tickers from the file instead of requiring them directly in the command. This makes the pipeline easier to scale, because the research universe can be changed by editing a text file rather than editing code.

For a larger version of the project, a file such as `config/tickers_sp500.txt` can be created with the S&P 500 ticker universe. Here, the limit argument requests the ten most recent annual reports per company. The metadata-output argument chooses the name of the CSV file that stores the filing metadata. The sleep-seconds argument controls how long the script waits between SEC requests. The max-retries argument controls how often the script retries temporary SEC errors before marking a filing as failed. These two options make the pipeline more robust for larger downloads, where occasional server errors or rate limits are expected. For a universe of roughly 500 companies, this command would produce approximately 5,000 filings before accounting for missing companies, amended filings, ticker changes, and firms with shorter public histories.

The `read_tickers()` function is responsible for reading the research universe. It can combine tickers passed directly through the command line with tickers stored in a file. It also normalizes the symbols to uppercase and removes duplicates. This keeps the rest of the script simple because the main download loop receives one clean list of tickers.

The `main()` function ties the full procedure together. It reads the command-line arguments, creates the SEC session, loads the ticker map, reads the ticker universe, loops through each company, downloads the selected filings, and collects one metadata row per filing. If a company or filing fails, the script does not stop the entire run. Instead, it records the failure in the metadata with `download_status` and `download_error`. This makes the pipeline easier to audit because successful and failed downloads are visible in one table.

The downloader creates two important outputs. The raw SEC filing documents are saved under:

```
data/raw/filings/
```

The filing metadata are saved in a CSV file such as:

data/interim/filings_metadata_sp500.csv

The metadata file records the ticker, CIK, accession number, filing date, report date, form type, SEC URL, and local file path. This table is the bridge between the raw EDGAR documents and the later stages of the pipeline. After the filings are downloaded, the next step is to clean the HTML text, extract textual features, and link each filing date to future stock returns.

4 Return Data and Label Construction

After the 10-K filings have been downloaded, the next step is to create financial outcome variables for each filing. These variables are called return labels because they describe what happened to the stock after the filing date. The script responsible for this step is:

Code/build_return_labels.py

The script reads the filing metadata CSV produced by the EDGAR downloader. Each row in that file represents one 10-K filing and contains the ticker, accession number, filing date, and local filing path. The filing date is the key event date: all return calculations are measured forward from this date, so the analysis does not use information that would only have been available later.

The command used to build the return labels is:

```
python Code/build_return_labels.py \
--filings data/interim/filings_metadata_sp500.csv \
--output data/processed/return_labels.csv \
--benchmark SPY \
--horizons 30 90 180 252
```

The argument `--filings` tells the script where to find the filing metadata. The argument `--output` chooses the file where the calculated return labels will be saved. The argument `--benchmark SPY` tells the script to use SPY as the market benchmark. The argument `--horizons 30 90 180 252` defines the forward trading-day windows over which returns and volatility are calculated.

The script first downloads adjusted daily closing prices using `yfinance`. Adjusted prices are used because they account for stock splits and dividends, which makes returns more comparable over time. Prices are downloaded separately for each stock ticker and once for the benchmark ticker. The benchmark price series is reused across all filings because the same market benchmark is needed for every company.

A filing date does not always fall on a trading day. For example, a filing can occur on a weekend or after the market has closed. To keep the procedure simple and reproducible, the script uses the first available trading day on or after the filing date as the starting price date. If the filing date is denoted by t , the starting price is therefore:

$$P_{t_0}^i,$$

where t_0 is the first trading day on or after the filing date for company i . For a horizon of h trading days, the ending price is:

$$P_{t_0+h}^i.$$

The forward return for company i over horizon h is calculated as:

$$R_h^i = \frac{P_{t_0+h}^i}{P_{t_0}^i} - 1.$$

The same calculation is applied to the benchmark:

$$R_h^m = \frac{P_{t_0+h}^m}{P_{t_0}^m} - 1,$$

where m denotes the market benchmark, in this project SPY. The excess return is then:

$$ER_h^i = R_h^i - R_h^m.$$

This value measures whether the stock outperformed or underperformed the market after the filing. The script also creates a binary outperformance label:

$$Outperform_h^i = \begin{cases} 1, & \text{if } ER_{i,h} > 0, \\ 0, & \text{if } ER_{i,h} \leq 0. \end{cases}$$

In addition to returns, the project calculates realized volatility. Realized volatility measures how much the stock price fluctuated during the forward window after the filing. First, the script computes daily returns within the window:

$$r_d^i = \frac{P_d^i}{P_{d-1}^i} - 1.$$

Then it calculates the standard deviation of these daily returns and annualizes it using the standard 252-trading-day convention:

$$\sigma_h^i = \sqrt{252} \cdot std(r_d^i).$$

This gives a future realized volatility label for each filing and each horizon. The interpretation is different from return prediction. A high realized volatility value does not necessarily mean the stock performed badly, it means the stock experienced more uncertainty or price movement after the 10-K filing. This is useful because the language in risk disclosures may be more strongly related to future uncertainty than to the direction of returns alone.

The final output of this script is:

`data/processed/return_labels.csv`

This file contains one row per filing. Important columns include the forward return columns for the 30, 90, 180, and 252 day horizons, the corresponding benchmark and excess return columns, the binary outperformance columns, and the realized volatility columns. These labels will later be merged with textual features extracted from the 10-K reports, producing the final modeling dataset.

5 Text Feature Extraction

Once the return labels have been created, the next step is to convert the raw 10-K filings into numerical text features. This step is handled by:

`Code/build_text_features.py`

The script reads the same filing metadata file used in the earlier pipeline steps:

`data/interim/filings_metadata_sp500.csv`

Each row in this metadata file contains a local path to a downloaded SEC filing. The text-feature script opens each local filing file, reads the raw HTML, and converts it into plain text. SEC filings are not clean text documents: they contain HTML tags, formatting instructions, tables, scripts, style definitions, and inconsistent spacing. The function `html_to_text()` removes the HTML structure and drops elements such as scripts, styles, and tables. The remaining text is then normalized so that repeated whitespace becomes a single space. The command for this step is:

```
python Code/build_text_features.py \
  --filings data/interim/filings_metadata_sp500.csv \
  --output data/processed/text_features.csv
```

The argument `--filings` tells the script where to find the EDGAR filing metadata, while `--output` specifies where the resulting feature table should be saved. The output of this step is:

```
data/processed/text_features.csv
```

The script also uses four separate dictionary files:

```
config/dictionaries/negative_words.csv
config/dictionaries/positive_words.csv
config/dictionaries/uncertainty_words.csv
config/dictionaries/litigious_words.csv
```

Each CSV file contains one column called `word`. These files make the textual analysis transparent, because the exact words used to construct the dictionary features can be inspected and changed without rewriting the feature extraction code. The negative dictionary contains words related to losses, risks, weakness, and deterioration. The positive dictionary contains words related to growth, gains, favorable outcomes, and improvement. The uncertainty dictionary contains words that indicate conditional or unclear future outcomes, such as whether something may, might, depend on, or fluctuate. The litigious dictionary contains words related to legal and regulatory disputes, such as claims, courts, settlements, and litigation.

After the filing has been converted to plain text, the script tokenizes it. In this context, tokenization means splitting the document into individual word units. The function `tokenize()` uses a regular expression to keep alphabetic word-like tokens and convert them to lowercase. This gives the script a consistent representation of the document that can be counted and compared across companies and years.

The core feature calculation is performed by `text_features()`. The first group of features measures document size and complexity. These include the total word count, the number of unique words, the type-token ratio, the average word length, the average sentence length, and the logarithm of the word count. The type-token ratio is defined as:

$$TTR_i = \frac{\text{number of unique words in filing } i}{\text{total number of words in filing } i}.$$

It measures lexical diversity. A higher type-token ratio means that the filing uses a wider variety of words relative to its length, while a lower value means that the same words are repeated more often. In the context of 10-K filings, this feature can be interpreted as a rough measure of how varied or repetitive the disclosure language is. These variables are useful because the length and complexity of a filing may itself reflect the amount of information, risk, or business complexity disclosed by a company.

The second group of features measures finance-related language. The script counts how often words from small finance-oriented dictionaries appear in the filing. It then divides those counts by the total number of words in the document, producing word-share variables. The main

variables are `negative_share`, `positive_share`, `uncertainty_share`, and `litigious_share`. These variables are interpretable: a higher uncertainty share, for example, means that a larger fraction of the filing is written using words associated with uncertainty. In the Python script, this is done by `load_dictionaries()`, which reads the four CSV files, and by `word_share()`, which calculates the fraction of all tokens that belong to each dictionary. Formally, for dictionary category c , the share is calculated as:

$$Share_{i,c} = \frac{\text{number of tokens in filing } i \text{ that appear in dictionary } c}{\text{total number of tokens in filing } i}.$$

This means that the resulting variables are normalized by document length. A longer filing does not automatically receive a higher negative or uncertainty score simply because it contains more words overall.

The script also adds year-over-year change features with `add_change_features()`. For each company, filings are ordered by filing date, and the script calculates how selected textual features changed relative to the previous filing. Examples include changes in negative language, uncertainty language, litigious language, and document length. These change features are important because many annual reports repeat large portions of prior-year text. A sudden increase in risk or uncertainty language may therefore be more informative than the absolute level of that language in a single filing.

The script records a `feature_status` for every filing. If the text is successfully extracted, the status is `ok`. If the local filing path is missing or the document cannot be processed, the status is `failed`. If the extracted document is unusually short, the status is marked as `suspicious`. This makes the feature table auditable: problematic filings can be inspected without stopping the entire pipeline.

The text-feature file is not the final dataset by itself. It only describes the language of each filing. The final modeling dataset is created after these textual features are merged with the return labels. That merge is done by:

`Code/build_model_dataset.py`

The script reads two input files: `text_features.csv` and `return_labels.csv`. It uses three merge keys to connect the two tables: `ticker`, `accession_number`, and `filing_date`. These keys identify the same company filing in both datasets. Before merging, the normalization function standardizes the ticker symbols, accession numbers, and filing dates so that small formatting differences do not prevent matching rows from joining correctly.

The function `require_columns()` checks that the necessary merge keys are present in both input files. If one of the required columns is missing, the script stops with a clear error message. This prevents the creation of an incorrect modeling table from incomplete inputs.

By default, the script keeps only rows where the text features have `feature_status = ok` and the return labels have `label_status = ok`. This removes filings where either the document text could not be processed or the stock-price labels could not be calculated. If failed rows should be kept for inspection, the optional command-line argument `--keep-failed` can be used.

The actual merge is performed with an inner join. This means that a filing only appears in the final modeling dataset if it exists in both the text-feature file and the return-label file. The script also uses one-to-one validation, which checks that each filing appears only once in each table. The output is saved as:

`data/processed/model_dataset.csv`

This final file contains one row per usable filing, with textual variables on the left side of the row and future return or volatility labels on the right side. It is the dataset used for the later modeling stage. For each company-year filing, the prediction targets include 30-trading-day, 90-trading-day, 180-trading-day, and 252-trading-day forward returns, excess returns relative

to SPY, binary outperformance indicators, and realized volatility over the same horizons. The main target in the first version is the 252-trading-day horizon, because it approximately matches a one-year investment period and fits the annual frequency of 10-K filings. Realized volatility is included as a separate target because corporate disclosure language may reveal future uncertainty even when it does not strongly predict the direction of returns.

6 Textual Features and Modeling

6.1 Textual Features and Modeling Approach

The project should start with interpretable textual features before moving to large language model or transformer-based features.

First, we will only use dictionary-based features. Finance-specific sentiment dictionaries, especially the Loughran-McDonald dictionary, are more appropriate than generic sentiment dictionaries because many words have different meanings in financial contexts. For example, “liability” and “tax” may be negative in common language but are normal financial terms.

Candidate features include negative word share, positive word share, uncertainty word share, litigious word share, modal strong and modal weak word share, report length, average sentence length, readability scores, and the number of risk-related terms.

Next, we consider change-based features, because firms often repeat large parts of their annual reports. Changes from one year to the next may be more informative than the level of sentiment in a single report. Important features include year-over-year change in negative tone, year-over-year change in uncertainty, textual similarity between the current and previous 10-K, new risk-factor language, and removed or shortened risk disclosures.

Lastly, we also consider embedding-based features. An embedding is a numerical vector representation of text. Instead of counting selected dictionary words, an embedding model maps a sentence, paragraph, or document section into a vector of numbers that is intended to preserve semantic meaning. Texts with similar meaning should have similar vectors, even if they do not use exactly the same words. In this project, embedding-based features would be a later extension after the interpretable baseline has been established. Financial language models such as FinBERT or general sentence-transformer models could be used to embed sections of the 10-K. Since 10-K reports are long documents, the report would first be split into smaller chunks. Each chunk would receive its own embedding, and the chunk embeddings could then be averaged, pooled by section, or summarized into document-level variables that enter the prediction model.

The modeling strategy follows the same logic as the feature construction: start with transparent, interpretable inputs and then increase complexity only when it adds useful information. The first benchmark is a naive baseline that predicts average historical excess return. After that, we compare linear models and tree-based models. This sequence is useful because each model answers a different question. The naive baseline asks whether the text model improves on a simple historical average. The linear model asks whether the textual features have a stable additive relationship with future outcomes. The tree-based model asks whether non-linear thresholds and interactions between features are useful.

Here the split between training and testing is kept quite simple: filings before 2023 are used for training, and filings from 2023 onward are used for testing. This keeps the evaluation closer to the real forecasting problem, where the model must be trained on past filings and evaluated on later filings.

6.2 Dependent Variables

The project uses three dependent variables, because the text in a 10-K filing may be related to different aspects of future stock behavior. The first dependent variable is a binary outperformance indicator:

$$Y_i^{class} = Outperform_{i,252}.$$

This variable equals one if stock i outperforms SPY over the 252 trading days after the filing date, and zero otherwise. It is used for the classification models. The research question behind this target is whether textual information in the filing helps distinguish firms that will beat the market from firms that will not.

The second dependent variable is the future excess return:

$$Y_i^{ret} = ER_{i,252}.$$

This is the stock's 252-day return minus the 252-day return of SPY. It is a continuous variable and is used for regression models. This target does not only ask whether the stock beats the benchmark, but also by how much.

The third dependent variable is future realized volatility:

$$Y_i^{vol} = \sigma_{i,252}.$$

This variable measures the volatility of daily stock returns over the following 252 trading days. It is also continuous. This target is included because risk, uncertainty, and litigation language may be more informative about future variation in returns than about the direction of returns.

6.3 Logistic Regression

Logistic regression is used for the outperformance classification task. The dependent variable is `outperformed_252d`, which takes the value one if the stock outperforms SPY over the next 252 trading days and zero otherwise. The independent variables are the textual features collected in X_i , such as document length, type-token ratio, negative word share, positive word share, uncertainty word share, litigious word share, and selected year-over-year changes in these features. The model estimates the probability that a filing belongs to the outperforming class:

$$\Pr(Y_i^{class} = 1 \mid X_i) = \frac{1}{1 + \exp[-(\beta_0 + X_i' \beta)]}.$$

The logistic function transforms the linear score $\beta_0 + X_i' \beta$ into a probability between zero and one. This is useful because the dependent variable is not a continuous return, but a yes-or-no outcome. The coefficients β are interpreted in terms of log-odds. For example, a positive coefficient on `uncertainty_share` would mean that, holding the other text features fixed, a higher share of uncertainty words is associated with a higher predicted probability of future outperformance. A negative coefficient would imply the opposite relationship.

Logistic regression is a useful first model because it is transparent. It does not automatically find complex interactions, but it makes the direction of each feature's relationship with the dependent variable easier to inspect. The model is not estimated by ordinary least squares, because the dependent variable is binary. Instead, logistic regression is estimated by maximum likelihood. The likelihood asks how probable the observed sequence of zeros and ones is under a given set of coefficients, and the algorithm chooses the coefficients that make the observed classifications most likely. For one filing, define:

$$p_i(\beta) = \Pr(Y_i^{class} = 1 \mid X_i) = \frac{1}{1 + \exp[-(\beta_0 + X_i' \beta)]}.$$

The likelihood for the full sample is:

$$L(\beta) = \prod_{i=1}^n p_i(\beta)^{Y_i^{class}} [1 - p_i(\beta)]^{1-Y_i^{class}}.$$

It is easier to maximize the log-likelihood:

$$\ell(\beta) = \sum_i \left[Y_i^{class} \log(p_i(\beta)) + (1 - Y_i^{class}) \log(1 - p_i(\beta)) \right],$$

so the logistic-regression estimator is:

$$\hat{\beta} = \arg \max_{\beta} \ell(\beta).$$

Unlike OLS, this estimator does not have a simple closed-form solution. The coefficients must be found numerically. Conceptually, the algorithm starts with an initial guess for β , calculates the log-likelihood and its slope, updates the coefficients in the direction that improves the objective, and repeats this process until the improvement becomes very small. In the Python code, this estimation is performed in:

`Code/train_baseline_models.py`

inside the function `train_classification()`. The model is not optimized manually in our own code. Instead, the script calls the built-in `LogisticRegression` estimator from scikit-learn's linear-model module. The model is created with 1,000 maximum iterations and balanced class weights, and it is wrapped in `linear_preprocessor_model()`. Balanced class weights mean that observations from the less frequent class receive more weight in the objective, so the model is not rewarded for simply favoring the majority class. Since the code does not specify a solver, `sklearn` uses its default logistic regression solver, `lbfgs`. L-BFGS is a quasi-Newton optimization method. The implemented `sklearn` model also uses the default L_2 penalty. Thus, in the actual code, the fitted coefficients come from a regularized version of the maximum-likelihood problem:

$$\hat{\beta} = \arg \max_{\beta} \left[\ell(\beta) - \lambda \sum_j \beta_j^2 \right].$$

In scikit-learn's logistic regression implementation, the penalty strength is controlled by the parameter `C`, which is the inverse of regularization strength. In the notation above, this means:

$$C = \frac{1}{\lambda} \quad \text{or equivalently} \quad \lambda = \frac{1}{C}.$$

A smaller `C` therefore means a larger λ and a stronger penalty, while a larger `C` means a smaller λ and a weaker penalty. Our code does not set `C` explicitly when creating the `LogisticRegression` model, so scikit-learn uses its default value `C=1.0`. In this baseline specification, this corresponds to a fixed regularization setting of $C = 1.0$, or equivalently $\lambda = 1.0$ in the simplified notation used here. Therefore, the penalty strength is not estimated from the data in this baseline model, instead it is fixed at the library default. In a later extension, `C` could be chosen by a validation set or a grid search over several candidate values. The penalty directly works by making large coefficients costly. It does not detect correlated features by itself. The connection to correlation is indirect: when two variables contain similar information, such as word count and log word count, several different coefficient combinations may fit the training data almost equally well. Without a penalty, the model can assign unstable coefficients by putting too much weight on one variable and too little on the other. Small changes in the training data can then lead to large changes in the estimated coefficients. The L_2 penalty discourages

this by preferring coefficient combinations with smaller overall magnitude. The pipeline first imputes missing feature values with the median, then standardizes the variables, and then fits the logistic regression using `model.fit(x_train, y_train)`. After fitting, `predict_proba()` calculates the predicted probability of future outperformance for each test filing. The standardization step is important because the linear model does not use the raw feature values directly. In `Code/train_baseline_models.py`, the function `linear_preprocessor_model()` builds the pipeline:

```
("imputer", SimpleImputer(strategy="median"))
("scaler", StandardScaler())
("model", model)
```

The imputer replaces missing values with the median value of the corresponding feature in the training data. The scaler then transforms each feature into a standardized variable:

$$X_{ij}^{std} = \frac{X_{ij} - \bar{X}_j}{s_j},$$

where $\bar{X}_j$ and s_j are the training-sample mean and standard deviation of feature j . Thus, for logistic regression, the variables that enter the model are standardized versions of the textual features, not the raw values. This matters because variables such as word count and uncertainty share are measured on very different scales.

6.4 Ridge Regression

The return and volatility regressions are linear regressions, but they are not simple OLS in the implemented code. A simple OLS model would choose coefficients only by minimizing the sum of squared prediction errors:

$$\min_{\beta} \sum_i (Y_i - \alpha - X_i' \beta)^2.$$

The project instead uses ridge regression for the linear regression benchmarks. Ridge regression keeps the same linear prediction form,

$$\hat{Y}_i = \alpha + X_i' \beta,$$

but adds a penalty for large coefficients:

$$\min_{\beta} \sum_i (Y_i - \alpha - X_i' \beta)^2 + \lambda \sum_j \beta_j^2.$$

The first part is the usual squared-error loss. The second part is the ridge penalty. Similar to before the penalty can help to have more stable coefficient estimates. If two predictors contain similar information, OLS may have difficulty separating which variable should receive the effect. This can produce coefficients that are large, unstable, or sensitive to small changes in the sample. Ridge regression addresses this by making large coefficients costly. It does not remove correlated variables, but it shrinks coefficients toward smaller values and often chooses a smoother coefficient combination among variables that fit the data similarly well.

In the code, the ridge models are fitted inside `train_regression()`. The model dictionary contains `ridge_regression`, which is created as:

```
linear_preprocessor_model(Ridge(alpha=1.0))
```

The value `alpha=1.0` is the ridge penalty strength λ . In this baseline version, λ is therefore not learned automatically, instead it is set to 1.0 in the code as a simple default choice. A larger value would shrink the coefficients more strongly, while a smaller value would make the model closer

to ordinary least squares. In a more extensive model-selection step, λ could be chosen by comparing several candidate values on a validation sample or by using cross-validation. The same function is used twice: once with `excess_return_252d` as the dependent variable and once with `realized_volatility_252d` as the dependent variable. In both cases, `model.fit(x_train, y_train)` estimates the coefficients on the training sample, and `model.predict(x_test)` produces the numerical predictions for the test sample. The script then evaluates these predictions with mean absolute error, root mean squared error, R^2 , and Spearman rank correlation. Ridge regression uses the same `linear_preprocessor_model()` function as logistic regression. This means that the ridge model is also fitted on imputed and standardized features. In this notation, `x_train` is the feature matrix for all usable filings before 2023. Each row represents one 10-K filing and each column represents one of the 16 textual variables used by the model, including word count, type-token ratio, the four dictionary word shares, and their selected year-over-year changes. The corresponding `y_train` is a one-dimensional target vector with one observed outcome for every row of `x_train`. For the excess-return model, these values are the observed `excess_return_252d` values and for the volatility model, they are the observed `realized_volatility_252d` values. In the current dataset, both 252-day regression tasks contain 2,226 training filings, so `x_train` has dimensions $2,226 \times 16$ and `y_train` contains 2,226 values. `x_test` contains the same 16 features, in the same column order, for the 1,397 usable filings from 2023 onward, and therefore has dimensions $1,397 \times 16$. The model learns the relationship between the features and outcomes from `x_train` and `y_train` and it then applies that learned relationship to `x_test`, whose future outcomes are not supplied to the model during prediction. The separate observed test outcomes are retained only afterward to measure prediction accuracy.

The coefficient β_j in the ridge equation therefore refers to a one-standard-deviation change in feature j , not to a one-unit change in the raw feature. By contrast, the random forest models use the separate `tree_model()` function, which only applies median imputation and does not standardize the features, because tree splits are based on feature thresholds and do not require variables to be on the same scale.

6.5 Random Forests

Random forests are used as the first non-linear machine learning model. More precisely, the project fits three separate random forest models. The `RandomForestClassifier` predicts whether a stock will outperform SPY, using `outperformed_252d` as its binary target. The first `RandomForestRegressor` predicts the continuous `excess_return_252d` target, and the second `RandomForestRegressor` predicts the continuous `realized_volatility_252d` target. These are three fitted forests, not one forest that produces all three outcomes simultaneously. Each forest contains 300 decision trees.

A random forest is an ensemble of many decision trees. Each tree repeatedly splits the sample into smaller groups based on feature thresholds. For example, one split might separate filings with high uncertainty language from filings with low uncertainty language, while another split might separate very long reports from shorter reports. A single decision tree can be unstable, so the random forest fits many trees and combines their predictions. If a forest contains B trees, each tree can be written as a prediction function $T_b(X_i)$, where $b = 1, \dots, B$. The final forest prediction is therefore not produced by one single tree, but by combining many tree-level predictions.

Here, X_i is the complete feature vector for filing i , meaning one row of the feature matrix supplied to the random forest. In this project, X_i contains the 16 textual variables for one 10-K filing: word count, unique word count, type-token ratio, average word length, average sentence length, log word count, the negative, positive, uncertainty, and litigious word shares, and the selected year-over-year changes in those measures. For example, $T_b(X_i)$ means that the feature values of filing i are passed through the split rules of tree b until they reach a final leaf, which

produces that tree’s prediction.

For regression, this combination is an average:

$$\hat{Y}_i = \frac{1}{B} \sum_{b=1}^B T_b(X_i).$$

Each tree gives its own prediction for the future excess return or future realized volatility, and the random forest uses the mean of these predictions. This averaging reduces the instability of individual decision trees.

For the classification task, the random forest classifier predicts the same dependent variable as logistic regression, `outperformed_252d`. Instead of estimating one linear equation, it estimates the probability of outperformance by averaging the votes of many trees. If $T_b(X_i)$ equals one when tree b predicts outperformance and zero otherwise, then the estimated probability is:

$$\hat{p}_i = \Pr(Y_i^{class} = 1 \mid X_i) = \frac{1}{B} \sum_{b=1}^B T_b(X_i).$$

The predicted class is then usually one if this probability is above a threshold such as 0.5, and zero otherwise. This allows the model to capture relationships such as uncertainty language mattering only when negative language is also high, or document length mattering only for firms with large year-over-year changes in tone.

As a simple example, suppose the forest contains 300 trees and we evaluate one filing from a company. The filing has a high uncertainty-word share, a long document length, and a large year-over-year increase in negative language. One tree may first split on `uncertainty_share`, if the value is high, it may then split on `negative_share_yoy_change`, if this value is also high, the tree may classify the filing as likely to underperform. Another tree may use a different route and first split on `log_word_count`, then on `litigious_share`. A third tree may find that positive language offsets some of the risk language. If 165 of the 300 trees vote for outperformance and 135 vote against it, then the predicted probability of outperformance is:

$$\hat{p}_i = \frac{165}{300} = 0.55.$$

Using a 0.5 threshold, the model would classify this stock as an expected outperformer. The important point is that the final prediction is not based on a single split or a single rule. It is based on the average decision of many different trees.

The trees are made different from each other through randomization. First, each tree is trained on a bootstrap sample, meaning that it receives a random sample of the training observations drawn with replacement. Second, for the classification forest, only a random subset of the available features is considered at each split. The code does not set `max_features` explicitly, so scikit-learn uses its classifier default of the square root of the total number of features. With 16 features, approximately four randomly selected features are therefore considered at each classification split. The regression forest has a different scikit-learn default: `max_features=1.0`. Consequently, all 16 features are considered at each regression split, although its trees still differ because they use random bootstrap samples. The forest combines these many different trees into a more stable prediction.

It is important to separate this random feature-selection step from the choice of a threshold. The candidate features are selected randomly in the classification forest, but the thresholds for those candidate features are not generated randomly. For each candidate feature, the standard decision-tree algorithm systematically examines possible split points derived from the observed training values in the current node. It then selects the feature-threshold combination that produces the greatest improvement in the split objective. The order in which features appear

in a tree is therefore not specified beforehand: at every node it results from a new candidate-feature set and a systematic comparison of the possible thresholds.

The training of one tree works recursively. At the beginning, the tree receives a training sample with textual features and the corresponding dependent variable. A node is one point in the tree that contains a subset of the training observations. The first node is called the root node and contains all observations used to train that tree. When the tree applies a split, this node is divided into two child nodes: a left child node for observations that satisfy the split rule, and a right child node for observations that do not. A node that is no longer split is called a terminal node or leaf. The prediction of a single tree is made at this final leaf. A leaf is therefore the endpoint of a path through the decision tree. For classification, the leaf stores the class proportions of the training observations that reached it, such as the share of filings that outperformed the S&P 500 benchmark, represented in the dataset by SPY. A class proportion is the fraction of observations in each possible category. In this project, class 1 means that the filing's stock subsequently outperformed the S&P 500 benchmark, while class 0 means that it did not. For example, suppose 10 training filings reach one leaf and 7 belong to class 1 while 3 belong to class 0. The unweighted class proportions in that leaf are:

$$p(\text{class} = 1) = \frac{7}{10} = 0.70, \quad p(\text{class} = 0) = \frac{3}{10} = 0.30.$$

Without class weights, a new filing that reaches this leaf receives a class-1 probability of 0.70 from this individual tree. Because 0.70 is above the classification threshold of 0.50, the tree predicts that the filing's stock will outperform the S&P 500 benchmark. The implemented classifier, however, uses `class_weight` set to `balanced_subsample`. This recalculates a weight for each class separately within the bootstrap sample used to train each tree. For class k , the weight is:

$$w_k = \frac{N}{KN_k},$$

where N is the number of observations in that tree's bootstrap sample, $K = 2$ is the number of classes, and N_k is the number of bootstrap observations belonging to class k . A less frequent class therefore receives a larger weight. These weights enter in two places. They are used when the tree calculates Gini impurity to select its splits, and they are used when the tree calculates the class probabilities stored in a leaf.

For example, suppose one tree's bootstrap sample contains 100 filings, of which 80 are non-outperformers and 20 are outperformers. The class weights are:

$$w_0 = \frac{100}{2 \cdot 80} = 0.625, \quad w_1 = \frac{100}{2 \cdot 20} = 2.5.$$

Now suppose the same leaf contains 3 non-outperformers and 7 outperformers. The weighted count for class 0 is $3 \cdot 0.625 = 1.875$, while the weighted count for class 1 is $7 \cdot 2.5 = 17.5$. The class-1 probability stored in the leaf is therefore:

$$p_w(\text{class} = 1) = \frac{17.5}{17.5 + 1.875} \approx 0.903.$$

In this weighted example, a new filing that reaches the leaf receives a 0.903 probability of outperformance from this tree, rather than the unweighted value of 0.70. The filing is then passed through the other 299 trees, whose bootstrap samples, class weights, and reached leaves may differ. The random forest averages the 300 weighted class-1 probabilities. If this final average is at least 0.50, the complete forest classifies the filing as an expected outperformer. Otherwise, it classifies it as an expected non-outperformer.

For regression, a target value is the observed numerical outcome that the model is trying to predict. It is `excess_return_252d` in the excess-return forest and `realized_volatility_252d`

in the volatility forest. Suppose four training filings reach one leaf of the excess-return tree and their observed future excess returns are -2 , 1 , 5 , and 8 percent. The leaf stores their average target value:

$$\bar{Y}_{leaf} = \frac{-0.02 + 0.01 + 0.05 + 0.08}{4} = 0.03.$$

Thus, if a new filing reaches this leaf, that tree predicts a 3 percent future excess return. In a volatility tree, the calculation is identical, except that the values being averaged are the observed future realized volatilities. When a new filing is sent through the tree, it follows the split rules until it reaches one leaf, and the class proportion or average target value stored in that leaf becomes the tree’s prediction. The complete random forest then combines this prediction with the predictions from its other trees.

In other words, parent and child nodes describe the direction of a split. The parent node is the group before the split, and the child nodes are the two groups after the split. For example, suppose a parent node contains 10 filings and the tree tests the rule:

$$\text{uncertainty_share} \leq 0.012.$$

The left child node contains all filings for which this rule is true. In the worked example below, this would be filings 1 to 5, because their uncertainty shares are less than or equal to 0.012. The right child node contains all filings for which the rule is false. In the same example, this would be filings 6 to 10, because their uncertainty shares are greater than 0.012. Therefore the split can be written as:

Parent node with 10 filings $\rightarrow$ Left child node with 5 filings + Right child node with 5 filings.

After this split, each child node can itself become a parent node if the tree splits it again. This is why the tree is built recursively: a group is split into smaller groups, and those smaller groups may be split again until the stopping rules are reached. The maximum length of a path is not determined by the number of available features, because the same feature can be used again at different nodes or at different thresholds. In this project, the length is instead restricted directly by `max_depth=5`. Thus, even though the dataset contains 16 features, a path from the root to a leaf contains at most five splits. This does not mean that the complete tree contains only five nodes. Scikit-learn assigns depth 0 to the root, so a tree with `max_depth=5` can have six node levels, from the root at depth 0 to leaves at depth 5. Because every decision node can create a left and a right child, a completely filled binary tree of depth 5 could theoretically contain:

$$1 + 2 + 4 + 8 + 16 + 32 = 63$$

nodes in total, including at most 32 leaves. The fitted trees will often contain fewer nodes because a branch can stop early when no useful split remains or when the minimum-leaf requirement of 10 observations prevents a further split. An individual filing does not visit all of these nodes. It follows only one root-to-leaf path and therefore passes through at most five split decisions.

At every internal decision node of the excess-return and realized-volatility regression trees, all 16 textual features form the candidate-feature set again. Each feature has a node-specific list of eligible thresholds based only on the training observations that reached that node. The algorithm compares all eligible feature–threshold pairs but implements only one selected pair as the node’s split rule. The two child nodes then repeat the same search with all 16 features, but generally with different observations and therefore different candidate thresholds. Different nodes may select different features, and the same feature may be selected again at another node with a different threshold.

At a node N , the algorithm does not begin with only one feature and one threshold. It begins with a set of candidate features. For the two regression forests in this project, this candidate set

contains all 16 features. Every feature j has its own set of eligible thresholds, denoted $\mathcal{C}_j(N)$, calculated from the values of that feature among the observations currently in node N . The complete search set can therefore be written as:

$$\mathcal{S}(N) = \{(j, c) : j \in \{1, \dots, 16\}, c \in \mathcal{C}_j(N)\}.$$

Here, N means the current node. The expression $\mathcal{S}(N)$ is the search set at node N , meaning the complete list of eligible feature–threshold combinations that the algorithm can test at that node. The symbol j identifies one of the 16 features. The symbol c identifies one eligible threshold for that feature. The smaller set $\mathcal{C}_j(N)$ contains all eligible thresholds for feature j , based on the observations currently in node N . Therefore, $\mathcal{S}(N)$ combines the threshold lists from all 16 features into one search set.

For example, suppose a simplified node has only two features. Feature 1 has candidate thresholds 0.0065 and 0.0105, while feature 2 has candidate thresholds 10,000 and 20,000. Its search set would be:

$$\mathcal{S}(N) = \{(1, 0.0065), (1, 0.0105), (2, 10,000), (2, 20,000)\}.$$

The algorithm loops over all four entries. For each entry, it temporarily divides the observations into a left and a right child and calculates the resulting improvement. In the actual regression trees, the same process uses all 16 features and all their eligible thresholds. This notation is not one vector of 16 thresholds applied simultaneously. It is a list of alternative single-feature splits that are compared with one another.

Important distinction. All 16 features are evaluated at a regression node, but they are evaluated separately as alternative candidates. One candidate split always contains exactly one feature and one threshold. For example, the algorithm can evaluate:

$$\text{uncertainty_share} \leq 0.012,$$

then separately evaluate:

$$\text{negative_share} \leq 0.008,$$

and then separately evaluate:

$$\text{word_count} \leq 40,000.$$

These are three competing one-feature splits. They are not combined into one simultaneous rule. After evaluating every eligible one-feature candidate, the node selects one feature and one threshold. The other features were alternatives during the search and do not appear in that node’s final rule. Several features can still affect a filing through a sequence of nodes. For example, the root can split on `negative_share`, and the child node reached by the filing can then split on `uncertainty_share`. Therefore, one node uses one feature, while one complete root-to-leaf path can use several features.

For the regression objective, the selected pair is:

$$(j^*, c^*) = \arg \max_{(j, c) \in \mathcal{S}(N)} \Delta SSE_N(j, c).$$

The expression $\Delta SSE_N(j, c)$ means the reduction in the Sum of Squared Errors produced at node N when feature j is split at threshold c . The algorithm chooses the pair with the largest reduction. Only after this systematic comparison does the node implement the single best rule:

$$X_{ij^*} \leq c^*.$$

Thus, many features and many thresholds are considered when choosing a split, but the final split at one node uses exactly one feature and one scalar threshold. In this project, X_{ij} could be a variable such as `uncertainty_share`, `negative_share`, or `log_word_count`. The thresholds are neither chosen by hand nor drawn randomly in the random forest used here. The model

systematically compares them and selects the feature–threshold pair that produces the greatest improvement in the objective.

More precisely, for a numerical feature the algorithm usually sorts the observed training values of that feature and evaluates candidate split points between neighboring values. It is therefore closer to a systematic search over the values that actually appear in the data than to a researcher manually defining a small grid. For example, if the observed values of `uncertainty_share` in a node are 0.004, 0.009, 0.012, and 0.020, then candidate thresholds can be placed between these values, such as 0.0065, 0.0105, and 0.016. For each candidate threshold, the algorithm temporarily splits the observations into a left group and a right group and calculates how much the split improves the objective. The best threshold is the one with the largest improvement. If no candidate threshold improves the node enough, or if a stopping rule is reached, the node becomes a final leaf.

Testing all eligible feature–threshold pairs increases the computation time, but the increase is not exponential. Suppose node N contains n_N training observations and the regression tree considers p features. A numerical feature can have at most $n_N - 1$ candidate split positions between consecutive observations. The maximum number of candidates at that node is therefore approximately:

$$p(n_N - 1).$$

This expression grows through multiplication. Doubling the number of features approximately doubles the feature-search work, while doubling the number of observations approximately doubles the number of candidate positions before sorting costs are considered.

For the current 252-day regression sample, the root node contains 2,226 training filings and all 16 features are considered. Before accounting for repeated feature values and the minimum-leaf restriction, the root can therefore have at most approximately:

$$16(2,226 - 1) = 35,600$$

feature–threshold candidates. The actual number can be smaller because repeated values do not create new divisions and a threshold is not eligible if it would leave fewer than 10 observations in either child.

The feature values must also be ordered so that neighboring split positions can be examined. Sorting n_N observations has a cost of approximately $O(n_N \log n_N)$ for one feature. A simplified description of the work at one node is therefore $O(pn_N \log n_N)$. After a split, the observations are divided between the child nodes. If one child contains 1,000 filings and the other contains 1,226, the next level still contains a total of:

$$1,000 + 1,226 = 2,226$$

filing occurrences. With the maximum depth fixed at $d = 5$, a useful depth-limited approximation for one tree is $O(dpn \log n)$, although the exact runtime depends on the sizes and balance of the nodes.

For classification, a good split is one that separates the observations into groups with different outperformance rates. For example, suppose a tree is considering a split on `uncertainty_share`. If filings below a threshold of 0.012 have an outperformance rate of 60 percent, while filings above the threshold have an outperformance rate of 42 percent, then the split is useful because it creates two groups with different class distributions. A common way to measure this is the Gini impurity:

$$Gini = 1 - \sum_{k=1}^K p_k^2,$$

where p_k is the share of observations in class k . For the binary outperformance target, the two classes are outperformer and non-outperformer. The tree chooses the split that creates the

largest reduction in impurity across the two child nodes.

The following small example shows the calculation with 10 simplified filings. The tree only considers one feature, `uncertainty_share`, and one binary outcome, `outperformed_252d`. A value of one means that the stock outperformed SPY over the next 252 trading days, and a value of zero means that it did not.

Filing	<code>uncertainty_share</code>	<code>outperformed_252d</code>
1	0.004	1
2	0.006	1
3	0.007	0
4	0.009	1
5	0.011	1
6	0.013	0
7	0.016	0
8	0.018	0
9	0.021	0
10	0.024	1

Before splitting, the node contains five outperformers and five non-outperformers. The parent-node Gini impurity is therefore:

$$Gini_{parent} = 1 - (0.5^2 + 0.5^2) = 0.5.$$

The algorithm then sorts the feature values and tests candidate thresholds between neighboring values. One candidate threshold is $c = 0.012$, which lies between 0.011 and 0.013. This split creates a left node with filings 1 to 5 and a right node with filings 6 to 10. In the left node, four of the five filings outperformed. The left-node Gini impurity is:

$$Gini_{left} = 1 - (0.8^2 + 0.2^2) = 0.32.$$

In the right node, one of the five filings outperformed and four did not. The right-node Gini impurity is:

$$Gini_{right} = 1 - (0.2^2 + 0.8^2) = 0.32.$$

Because both child nodes contain five observations, the weighted impurity after the split is:

$$Gini_{split} = \frac{5}{10} \cdot 0.32 + \frac{5}{10} \cdot 0.32 = 0.32.$$

The impurity reduction from this candidate split is:

$$0.50 - 0.32 = 0.18.$$

The tree performs this calculation for many possible thresholds and selects the threshold with the largest impurity reduction. If this split is selected, a new filing with `uncertainty_share`=0.010 would go to the left node because $0.010 \leq 0.012$. The predicted probability of outperformance in that leaf would be the share of outperformers in the leaf:

$$\hat{p}_{leaf} = \frac{4}{5} = 0.80.$$

For a single tree, this means the filing would be classified as an outperformer if the classification threshold is 0.5. In a random forest, this is only one tree's contribution. The final probability is obtained by averaging this kind of leaf prediction across all trees.

For regression, the logic is similar, but the target is continuous. Instead of separating outperformers from non-outperformers, the tree chooses thresholds that reduce the variation of

the target within each group. A split is useful if the future excess returns or future realized volatilities inside each child node are more similar to each other than they were in the parent node. In practice, the tree chooses the split that most reduces squared prediction error. The abbreviation *SSE* means Sum of Squared Errors. For a node N , it is:

$$SSE(N) = \sum_{i \in N} (Y_i - \bar{Y}_N)^2.$$

In this formula, Y_i is the observed target value for filing i . It is the 252-day excess return in the excess-return forest and the 252-day realized volatility in the volatility forest. The term $\bar{Y}_N$, called Y bar for node N , is the arithmetic mean of the target values belonging to the training filings currently in node N :

$$\bar{Y}_N = \frac{1}{|N|} \sum_{i \in N} Y_i.$$

Here, $|N|$ is the number of training filings in node N . In the excess-return forest, $\bar{Y}_N$ is the average 252-day excess return of those filings. In the realized-volatility forest, it is their average 252-day realized volatility. For example, if a node in the excess-return tree contains four filings with target values of 0, 2, 8, and 10 percent, then:

$$\bar{Y}_N = \frac{0.00 + 0.02 + 0.08 + 0.10}{4} = 0.05.$$

Therefore, $\bar{Y}_N$ equals a 5 percent average excess return for that node.

For a candidate split into left and right child nodes, the improvement is:

$$\Delta SSE = SSE(N_{parent}) - [SSE(N_{left}) + SSE(N_{right})].$$

The algorithm systematically calculates this improvement for the candidate feature–threshold combinations and selects the combination with the largest positive ΔSSE . This is what “greatest improvement” means for both the excess-return forest and the realized-volatility forest. The prediction in a final leaf is the average target value of the training observations that fall into that leaf.

For example, suppose a regression node contains four filings with future excess returns of 0, 2, 8, and 10 percent. Their parent-node mean is 5 percent, so:

$$SSE_{parent} = (0.00 - 0.05)^2 + (0.02 - 0.05)^2 + (0.08 - 0.05)^2 + (0.10 - 0.05)^2 = 0.0068.$$

Suppose the rule `uncertainty_share` ≤ 0.012 places the filings with returns of 0 and 2 percent in the left child and those with returns of 8 and 10 percent in the right child. The child-node means are 1 and 9 percent, respectively, and their combined error is:

$$SSE_{children} = [(0.00 - 0.01)^2 + (0.02 - 0.01)^2] + [(0.08 - 0.09)^2 + (0.10 - 0.09)^2] = 0.0004.$$

The improvement produced by this candidate split is therefore:

$$\Delta SSE = 0.0068 - 0.0004 = 0.0064.$$

The tree compares this value with the improvements produced by the other candidate thresholds and features. It selects this split only if its reduction of 0.0064 is the largest available reduction. The realized-volatility forest performs exactly the same calculation, but its Y_i values are future realized volatilities rather than future excess returns.

As a small regression example, suppose one final leaf contains 10 training filings after a sequence of splits. Their average future 252-day excess return is 4 percent. If a new filing follows the same path through that tree and lands in this leaf, that tree predicts:

$$T_b(X_i) = 0.04.$$

If three trees in a small illustrative forest predict 4 percent, -1 percent, and 7 percent, the forest prediction would be:

$$\hat{Y}_i = \frac{0.04 - 0.01 + 0.07}{3} = 0.033.$$

This means that the forest predicts a 3.3 percent future excess return for this filing. In the actual code the same logic is used, but with 300 trees rather than three.

For the regression tasks, the random forest regressor predicts the 252-day excess-return target and the 252-day realized-volatility target. In these cases, each tree gives a numerical prediction, and the forest averages those predictions. The advantage is flexibility: the model can learn non-linear thresholds and interactions without manually specifying them. The disadvantage is that the model is less directly interpretable than logistic or ridge regression and can fit noise when the signal in the text features is weak.

In the Python code, the random forest models are defined in the baseline-model training script. The helper function `tree_model()` creates a `sklearn` pipeline with two steps. The first step is `SimpleImputer(strategy="median")`, which replaces missing feature values with the median value of that feature in the training data. This is especially useful for year-over-year change variables, because the first available filing for a company does not have a previous filing from which a change can be calculated. The second step is the random forest model itself.

For the outperformance classification task, the function `train_classification()` creates a `RandomForestClassifier`. In this project, the classifier uses `n_estimators=300`, meaning that the forest contains 300 trees. The argument `max_depth=5` limits the depth of each tree, so no tree can keep splitting indefinitely. The argument `min_samples_leaf=10` requires at least 10 observations in each final leaf, which prevents the model from creating extremely specific rules based on only one or two filings. The argument `class_weight="balanced_subsample"` adjusts for imbalance in the classification target within each bootstrap sample. After fitting the model, the code uses `predict_proba()` to obtain the predicted probability of outperformance and then classifies the filing as an outperformer if that probability is at least 0.5.

For the excess-return and volatility tasks, `train_regression()` creates a random forest regressor. It uses the same basic structure: 300 trees, a maximum depth of 5, a minimum leaf size of 10, and `random_state=42` so that the results are reproducible. Instead of predicted probabilities, the regressor returns numerical predictions with `predict()`. The code then evaluates these predictions with mean absolute error, root mean squared error, R^2 , and Spearman rank correlation, before saving the filing-level predictions and model metrics to the model-results folder.

6.6 Baseline Models, Code Execution, and Evaluation Metrics

This subsection focuses on how the baseline-model script executes the experiment and how its output metrics should be interpreted. The definitions of the dependent variables and the statistical models are not repeated here, because they were explained in the preceding subsections. The script is:

Code/train_baseline_models.py

The first experiment is executed with:

```
python Code/train_baseline_models.py \
    --dataset data/processed/model_dataset.csv \
    --output-dir reports/model_results \
    --test-year 2023 \
    --horizon 252
```

The function `main()` coordinates the complete workflow. It reads the command-line arguments, creates the output directory, calls `prepare_data()`, runs the classification and regression functions, and writes the metric and prediction files. The argument `--horizon 252` selects the three 252-trading-day targets. The argument `--test-year 2023` assigns filings before 2023 to training and filings from 2023 onward to testing.

The function `prepare_data()` reads `model_dataset.csv`, derives `filing_year` from the filing date, constructs the target-column names for the selected horizon, and verifies that the required targets and textual features exist. The feature names are stored once in `FEATURE_COLUMNS`. This ensures that every model receives the same ordered set of explanatory variables.

The function `time_split()` performs the split separately for each target. It first drops rows where that target is missing and then divides the remaining filings by year. This target-specific step is necessary because a recent filing can have valid text features but may not yet have 252 trading days of subsequent price data. In the current experiment, each task has 2,226 training rows and 1,397 test rows.

The function `train_classification()` constructs `x_train`, `y_train`, `x_test`, and `y_test`, fits both classifiers, and calls `predict_proba()` for the test filings. It saves each filing's predicted probability and its class prediction. The function `train_regression()` performs the equivalent workflow for one continuous target. It is called once for excess return and once for realized volatility. Each call fits both regression estimators, generates test predictions, and calculates the regression metrics.

The preprocessing pipelines are fitted only on the training data. Median imputation therefore uses training-sample medians, and standardization uses training-sample means and standard deviations. The fitted transformations are then applied to the test features. This sequencing prevents information from the test period from entering the training process.

The script also creates simple reference predictions. For classification, it identifies the majority class in the training sample and predicts that class for every test filing. For regression, it predicts the training-sample mean of the target for every test filing. These references are important because a model should be compared with a simple rule, not judged only by the absolute size of its own metric.

In this project, goodness of fit refers primarily to out-of-sample predictive performance on the test filings. A model that fits the training observations closely but performs poorly on later filings is not considered a good model. The metrics below are therefore calculated from `y_test` and the predictions produced from `x_test`.

Classification Metrics

Accuracy is the fraction of test filings assigned to the correct class:

$$Accuracy = \frac{\text{number of correct class predictions}}{\text{number of test filings}}.$$

Its range is zero to one, and a larger value is better. Accuracy must be read together with the majority-class baseline. A baseline is a simple reference prediction used to determine whether the fitted models improve on a rule that does not use any textual features. In this project, the training data determine which class the baseline predicts, while the test data determine the reported baseline accuracy.

First, the code calculates the average binary target in the training sample:

$$\bar{y}_{train} = \frac{\text{number of training outperformers}}{\text{number of training filings}}.$$

It then selects one majority-class prediction:

$$\hat{y}^{baseline} = \begin{cases} 1 & \text{if } \bar{y}_{train} \geq 0.5, \\ 0 & \text{if } \bar{y}_{train} < 0.5. \end{cases}$$

This class is selected once from `y_train` and is predicted for every test filing. The baseline is not fitted or evaluated on the training data alone. Its rule comes from the training data, but its reported accuracy is calculated against `y_test`. In the current training sample, 1,195 of 2,226 filings are outperformers, so:

$$\bar{y}_{train} = \frac{1,195}{2,226} \approx 0.537.$$

The baseline therefore predicts class 1, expected outperformance, for every test filing. In the test sample, 537 of 1,397 filings actually outperform. The baseline accuracy is consequently:

$$\text{Baseline accuracy} = \frac{537}{1,397} \approx 0.384.$$

The value 0.384 is therefore an out-of-sample test accuracy produced by a constant class chosen from the training sample. A fitted model should be compared with this reference because accuracy can otherwise appear favorable when one class is common.

ROC-AUC is calculated from the predicted outperformance probabilities for the test filings. For one classification threshold t , filing i is predicted as an outperformer when:

$$\hat{p}_i \geq t.$$

The words positive and negative refer to the class predicted by the model. Positive means predicted outperformance and negative means predicted non-outperformance. True and false indicate whether that prediction agrees with the observed test target. The four possible outcomes at threshold t are:

	Predicted outperformer	Predicted non-outperformer
Actual outperformer	True positive, $TP(t)$	False negative, $FN(t)$
Actual non-outperformer	False positive, $FP(t)$	True negative, $TN(t)$

A true positive is an actual outperformer whose predicted probability is at least t . A false negative is an actual outperformer whose probability is below t . A false positive is an actual non-outperformer whose probability is at least t . A true negative is an actual non-outperformer whose probability is below t . These four counts always add up to the number of test filings:

$$TP(t) + FN(t) + FP(t) + TN(t) = n_{test}.$$

The true-positive rate is:

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)},$$

where $TP(t) + FN(t)$ is the total number of actual outperformers. Therefore, $TPR(t)$ is the fraction of actual outperformers correctly identified at threshold t . The false-positive rate is:

$$FPR(t) = \frac{FP(t)}{FP(t) + TN(t)},$$

where $FP(t) + TN(t)$ is the total number of actual non-outperformers. Therefore, $FPR(t)$ is the fraction of actual non-outperformers incorrectly classified as outperformers.

For example, suppose a simplified test sample contains 100 filings. Forty are actual outperformers and 60 are actual non-outperformers. At threshold $t = 0.60$, suppose the model produces:

$$TP = 30, \quad FN = 10, \quad FP = 15, \quad TN = 45.$$

The rates for this threshold are:

$$TPR(0.60) = \frac{30}{30 + 10} = 0.75$$

and:

$$FPR(0.60) = \frac{15}{15 + 45} = 0.25.$$

The threshold 0.60 therefore produces the ROC point (0.25, 0.75).

The complete ROC curve requires more than one threshold. The algorithm sorts the distinct predicted probabilities and moves t from above the largest probability to below the smallest probability. When t is above every probability, no filing is predicted as an outperformer and the curve begins at (0, 0). Each time the threshold passes a predicted probability, one or more filings change from predicted non-outperformer to predicted outperformer. The four counts, TPR , and FPR are recalculated. When t is below every probability, every filing is predicted as an outperformer and the curve ends at (1, 1). Joining all points $(FPR(t), TPR(t))$ creates the receiver operating characteristic, or ROC, curve.

ROC-AUC is the area under this curve:

$$ROC-AUC = \int_0^1 TPR(FPR) dFPR.$$

Scikit-learn calculates the area numerically from consecutive ROC points. If the points are indexed in increasing order of FPR , the trapezoidal calculation can be written as:

$$ROC-AUC \approx \sum_{k=1}^K \frac{TPR_k + TPR_{k-1}}{2} (FPR_k - FPR_{k-1}).$$

Each term is the area of one trapezoid below a segment of the ROC curve, and their sum is the complete area. An equivalent interpretation is the probability that a randomly selected actual outperformer receives a higher predicted probability than a randomly selected actual non-outperformer. Its interpretation is:

$$ROC-AUC = \begin{cases} 1.0 & \text{perfect ranking,} \\ 0.5 & \text{no ranking ability,} \\ < 0.5 & \text{ranking is worse than random.} \end{cases}$$

A larger ROC-AUC is better. It is the primary classification metric in this project because it evaluates probability rankings without depending on only the fixed 0.5 classification threshold. The logistic-regression value of 0.514 means that a randomly selected outperformer is ranked above a randomly selected non-outperformer approximately 51.4 percent of the time.

Model	ROC-AUC	Accuracy	Baseline accuracy
Logistic regression	0.514	0.540	0.384
Random forest	0.491	0.482	0.384

Table 1: Out-of-sample classification metrics for 252-day outperformance.

The logistic regression ROC-AUC of 0.514 is only slightly above the no-ranking value of 0.5. The random forest ROC-AUC of 0.491 is slightly below 0.5. Although both model accuracies exceed the reported majority-class reference, ROC-AUC shows that their probability rankings contain little directional information.

Regression Metrics

Mean absolute error is the average absolute distance between the observed target Y_i and prediction $\hat{Y}_i$:

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|.$$

A smaller MAE is better. It has the same units as the target. Because returns and volatility are stored as decimals, an excess-return MAE of 0.289 means an average absolute prediction error of approximately 28.9 percentage points. The code also reports `baseline_mae`. As with baseline accuracy, the training data determine the reference prediction and the test data determine the reported error. For target Y , the code first calculates:

$$\bar{Y}_{train} = \frac{1}{n_{train}} \sum_{i \in train} Y_i.$$

It then uses this one training-sample mean as the prediction for every test filing:

$$\hat{Y}_i^{baseline} = \bar{Y}_{train} \quad \text{for every } i \in test.$$

Important distinction. The code does not calculate the mean of the test targets and does not use $\bar{Y}_{test}$ as the baseline prediction. The baseline value is fixed before the test outcomes are examined:

$$\underbrace{\bar{Y}_{train}}_{\text{used as the prediction}} \quad \text{not} \quad \underbrace{\bar{Y}_{test}}_{\text{not used as the prediction}}.$$

When the evaluation is performed, the observed test targets are available, so their mean $\bar{Y}_{test}$ can certainly be calculated. The reason it is not used here is that the code defines the baseline as an out-of-sample forecasting rule. The constant prediction must therefore be chosen from the training data and then carried unchanged into the test period. If the test mean were used instead, the constant would be selected after seeing all test outcomes. It could be reported as a post-hoc descriptive reference, but it would not represent a prediction that could have been made before the test period. For mean squared error, the test mean is also the constant that produces the smallest possible squared error on that test sample, so it is an especially favorable retrospective comparison. Calculating $\bar{Y}_{test}$ is not by itself leakage. The important point is that this project does not use it to construct the out-of-sample baseline prediction.

Finally, it compares these constant predictions with the observed test targets:

$$Baseline\ MAE = \frac{1}{n_{test}} \sum_{i \in test} |Y_i - \bar{Y}_{train}|.$$

The baseline MAE is therefore an out-of-sample test error. The value predicted for every test filing comes from the training sample, but the differences used to calculate the error come from the test sample. For excess return, the training-sample mean is approximately 0.037, and applying this prediction to the test filings produces a baseline MAE of 0.288. For realized volatility, the training-sample mean is approximately 0.329, and its test baseline MAE is 0.096. A fitted model improves on the reference only when its MAE is lower than the corresponding baseline MAE.

Root mean squared error is:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}.$$

A smaller RMSE is better. Squaring the errors gives unusually large mistakes more influence than they receive under MAE. Comparing RMSE with MAE therefore helps reveal whether a model makes some particularly large prediction errors.

Spearman rank correlation measures whether filings with larger predicted values also tend to have larger observed values. It is the ordinary correlation between the ranks of Y_i and $\hat{Y}_i$:

$$\rho_s = Corr\left(Rank(Y_i), Rank(\hat{Y}_i)\right).$$

Its range is minus one to one. A value near one indicates a strong correct ranking, zero indicates little monotonic ranking relationship, and a negative value indicates an inverse ranking. A larger positive value is better. This metric is useful when the ordering of firms matters even if the exact numerical predictions remain noisy.

Target	Model	MAE	Baseline MAE	RMSE	ρ_s
Excess return	Ridge	0.289	0.288	0.449	-0.019
Excess return	Random forest	0.292	0.288	0.452	-0.054
Volatility	Ridge	0.104	0.096	0.129	0.012
Volatility	Random forest	0.101	0.096	0.125	0.132

Table 2: Out-of-sample regression metrics. Lower MAE and RMSE are better, while higher Spearman correlation is better.

Neither excess-return model improves on the baseline MAE of 0.288, and both have negative Spearman correlation. The volatility models also have MAE values above their baseline MAE of 0.096. The random forest volatility model has the strongest positive rank correlation, at 0.132, but this remains modest. Taken together, the metrics show that the first-pass textual features do not yet provide strong out-of-sample predictions.

The code writes the combined metric table to:

`reports/model_results/baseline_model_results.csv`

It also writes separate filing-level prediction files for classification, excess return, and volatility. The file `feature_target_correlations.csv` contains Pearson and Spearman correlations between individual features and targets. These outputs separate summary evaluation from filing-level predictions, which makes later model comparisons reproducible.

References

- [1] Tim Loughran and Bill McDonald. *When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks*. Journal of Finance, 66(1):35–65, 2011.
- [2] Narasimhan Jegadeesh and Di Wu. *Word Power: A New Approach for Content Analysis*. Journal of Financial Economics, 110(3):712–729, 2013.
- [3] Lauren Cohen, Christopher Malloy, and Quoc Nguyen. *Lazy Prices*. Journal of Finance, 75(3):1371–1415, 2020.